SOLEWISE · FOOTWEAR INTELLIGENCE PLATFORM

Engineering Architecture, Problem Analysis & Technical Solution Whitepaper

Status: Production Live
Stack: FastAPI + React + PostgreSQL
Live URL: sole-wise.onrender.com

Executive Summary: Solewise was engineered to solve footwear manufacturing cost overruns by replacing fragmented spreadsheets with an intelligent, dual-specification material alternative and batch yield optimizer. This report outlines the core failure points in legacy footwear costing tools and details the end-to-end technical solutions implemented across database architecture, modern UI design, and cloud persistence.

60.7%	100%	13 / 13	33.1 kB
Direct Cost Reduction (Synthetic vs Leather)	Permanent Cloud Sync (Render PostgreSQL)	Automated Tests Passing (Zero DB Contamination)	Optimized CSS Bundle (Lightweight PWA Engine)

1. Identified Problems & Root Causes

A. Ephemeral Cloud Data Loss: Deployments on free-tier containers (Render) rely on non-persistent file storage. When services idle after 15 minutes, container restarts reverted the local SQLite (shoes_wise.db) to the base Git commit, wiping user-entered materials.

B. Corrupted Excel & CSV Exports: Raw CSV exports suffered from missing UTF-8 BOM headers and unformatted string prefixes, rendering garbled currency symbols (â‚¹) and numerical overflow errors (########) in MS Excel.

C. Database Fixture Contamination: Automated unit test suites were posting dummy test records (Full Grain Bovine...) directly against the active SQLite production database.

D. Desktop-Biased & Bloated UI: Complex navigation modules (Costing, Sequence, Suppliers) and hardcoded parameters (MOQ/Lead Time) overwhelmed mobile screens without responsive touch targets.

2. Implemented Engineering Solutions

A. Managed PostgreSQL Architecture: Decoupled storage from the app container. Updated SQLAlchemy engine with dynamic connection pooling (pool_pre_ping, pool_recycle) and integrated Render PostgreSQL for permanent, real-time persistence.

B. Native SheetJS (.xlsx) Engine: Replaced broken CSV outputs with an in-browser binary Excel generator (xlsx), applying dynamic column auto-fitting, typed numeric cells, and clean INR currency formatting.

C. Isolated In-Memory Test Harness: Refactored Pytest harness to intercept FastAPI dependencies with an isolated sqlite:///:memory: StaticPool, preserving complete production database purity.

D. Claude Design & PWA Ergonomics: Redesigned with Anthropic Claude aesthetic (warm parchment #FAF9F5, terracotta #D97757), slide-out mobile drawer, standalone PWA manifest, and custom squircle icons.

3. Technical Architecture & Live Interface Proof

Layer	Technology	Key Implementation
Frontend	React 18 + Vite + Tailwind	Claude aesthetic, Touch-friendly drawer, SheetJS Exc
Backend	FastAPI (Python 3.14)	REST API, SQLAlchemy 2.0 ORM, Financial simulatic
Database	Render Cloud PostgreSQL	Permanent ACID storage, automatic schema migratio
Deployment	Render Cloud Web Service	Full-stack single container serving SPA & API under o

